

CO4_AT3 - INDUSTRY PROBLEM SOLVING TASK

Vehicle Rental Pricing Engine

Name: CHINTHA ABHILASH REDDY

Register No: 192525462

Question: Q1 - Vehicle Rental Pricing Engine (50 Marks)

Objective: Develop a modular Python 3 system for Car, Bike, Truck and Electric vehicle rentals. Pricing rules are loaded from an external CSV file, rental charges are calculated from duration and distance, transactions are persisted, invalid inputs are handled using custom exceptions, and output files are generated.

Technology: Python 3.x standard library only (csv, json, datetime, abc). No third-party libraries are required to run the submitted program.

1. Problem Analysis and Design

The company rents Cars, Bikes and Trucks. Manual pricing may create inconsistent results. The proposed solution uses object-oriented design so each vehicle category is represented by a subclass. Rate information is not hard-coded in the processing logic; it is read from pricing_rules.csv, so a manager can update rates without modifying Python code.

System Flow

1) Load configuration -> 2) Read and validate user input -> 3) Create the correct rental subclass -> 4) Calculate charge -> 5) Save transaction -> 6) Generate receipt/report/log.

Class Design

Class / Module	Purpose
VehicleRental (abstract base class)	Stores common rental data, validates inputs, and provides shared cost calculation.
CarRental	Vehicle category: Car. Overrides vehicle_type.
BikeRental	Vehicle category: Bike. Overrides vehicle_type.
TruckRental	Vehicle category: Truck. Overrides vehicle_type.
ElectricVehicleRental	New category added without changing existing subclasses; demonstrates extensibility.
config_loader.py	Reads and validates external CSV pricing rules.
main.py	Input, object creation, calculations, file persistence, receipts, reports and logs.

Pricing Formula

Base amount = days x base_per_day + distance x per_km. Diesel can apply the configured fuel multiplier; Electric fuel receives a 5% efficiency reduction. During Peak season the configured season multiplier is applied. The final amount is rounded to two decimal places.

2. External Configuration / Rate Loader - Sub-task (b)

The `pricing_rules.csv` file contains category-specific rates. Because it is external, prices can be changed without editing the Python source code.

```
vehicle_type,base_per_day,per_km,fuel_multiplier,season_multiplier
Car,1200,12,1.00,1.10
Bike,500,5,1.00,1.05
Truck,2500,25,1.15,1.20
Electric,1500,10,0.90,1.08
```

`config_loader.py` validates that all required columns exist, converts numeric values to float, reports invalid values through `ConfigurationError`, and checks for a missing or empty configuration file.

`config_loader.py`

```
import csv

class ConfigurationError(Exception):
    pass

def load_pricing_rules(filename="pricing_rules.csv"):
    rules = {}
    try:
        with open(filename, newline="", encoding="utf-8") as f:
            reader = csv.DictReader(f)
            required = {"vehicle_type", "base_per_day", "per_km", "fuel_multiplier", "season_multiplier"}
            if not required.issubset(reader.fieldnames or []):
                raise ConfigurationError("Pricing CSV has missing columns")

            for row in reader:
                try:
                    rules[row["vehicle_type"]] = {
                        "base_per_day": float(row["base_per_day"]),
                        "per_km": float(row["per_km"]),
                        "fuel_multiplier": float(row["fuel_multiplier"]),
                        "season_multiplier": float(row["season_multiplier"])
                    }
                except (ValueError, KeyError) as e:
                    raise ConfigurationError("Invalid value in pricing CSV") from e
    except FileNotFoundError as e:
        raise ConfigurationError(f"Pricing file not found: {filename}") from e

    if not rules:
        raise ConfigurationError("Pricing file is empty")
    return rules
```

3. Base Class, Subclasses and Extensibility - Sub-tasks (a) and (f)

VehicleRental is an abstract base class. The subclasses override vehicle_type. This design makes the system easy to extend: ElectricVehicleRental was added as a new category without rewriting CarRental, BikeRental or TruckRental.

vehicle_rental.py

```
from abc import ABC, abstractmethod

class InvalidRentalInputError(Exception):
    pass

class VehicleRental(ABC):
    def __init__(self, days, distance, fuel_type, season, rules):
        self.days = days
        self.distance = distance
        self.fuel_type = fuel_type
        self.season = season
        self.rules = rules
        self.validate()

    def validate(self):
        if self.days <= 0:
            raise InvalidRentalInputError("Duration must be greater than 0 days")
        if self.distance < 0:
            raise InvalidRentalInputError("Distance cannot be negative")
        if self.fuel_type not in ("Petrol", "Diesel", "Electric"):
            raise InvalidRentalInputError("Fuel type must be Petrol, Diesel, or Electric")
        if self.season not in ("Normal", "Peak"):
            raise InvalidRentalInputError("Season must be Normal or Peak")

    @property
    @abstractmethod
    def vehicle_type(self):
        pass

    def calculate_cost(self):
        r = self.rules[self.vehicle_type]
        amount = self.days * r["base_per_day"] + self.distance * r["per_km"]
        if self.fuel_type == "Diesel":
            amount *= r["fuel_multiplier"]
        elif self.fuel_type == "Electric":
            amount *= 0.95
        if self.season == "Peak":
            amount *= r["season_multiplier"]
        return round(amount, 2)

class CarRental(VehicleRental):
    vehicle_type = "Car"

class BikeRental(VehicleRental):
    vehicle_type = "Bike"

class TruckRental(VehicleRental):
    vehicle_type = "Truck"

class ElectricVehicleRental(VehicleRental):
    vehicle_type = "Electric"
```

4. Core Processing, Persistence and Exceptions - Sub-tasks (c), (d), (e)

The main module reads user input, constructs the correct object, calculates the charge, and writes multiple output files. Custom exceptions reject invalid duration, distance, fuel type, season, vehicle category, customer name, and configuration errors. ValueError handles invalid numeric input, while OSError handles file problems.

main.py

```
import csv
import json
from datetime import datetime
from config_loader import load_pricing_rules, ConfigurationError
from vehicle_rental import (
    CarRental, BikeRental, TruckRental, ElectricVehicleRental,
    InvalidRentalInputError
)

TRANSACTION_FILE = "rental_transactions.csv"
LOG_FILE = "system_log.txt"
RECEIPT_FILE = "receipt.txt"
REPORT_FILE = "summary_report.json"

def write_log(message):
    with open(LOG_FILE, "a", encoding="utf-8") as f:
        f.write(f"{datetime.now().isoformat(timespec='seconds')} - {message}\n")

def save_transaction(data):
    fields = ["time", "customer", "vehicle", "days", "distance", "fuel", "season", "cost"]
    try:
        with open(TRANSACTION_FILE, "r", encoding="utf-8"):
            exists = True
    except FileNotFoundError:
        exists = False

    with open(TRANSACTION_FILE, "a", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=fields)
        if not exists:
            writer.writeheader()
        writer.writerow(data)

def write_receipt(data):
    text = (
        "VEHICLE RENTAL RECEIPT\n"
        "-----\n"
        f"Customer : {data['customer']}\n"
        f"Vehicle   : {data['vehicle']}\n"
        f"Days      : {data['days']}\n"
        f"Distance  : {data['distance']} km\n"
        f"Fuel      : {data['fuel']}\n"
        f"Season    : {data['season']}\n"
        f"Total     : Rs. {data['cost']:.2f}\n"
    )
    with open(RECEIPT_FILE, "w", encoding="utf-8") as f:
        f.write(text)

def write_report(data):
    report = {
        "title": "Vehicle Rental Pricing Engine - Transaction Summary",
        "generated_at": datetime.now().isoformat(timespec="seconds"),
        "transaction": data,
        "status": "success"
    }
    with open(REPORT_FILE, "w", encoding="utf-8") as f:
        json.dump(report, f, indent=4)

def create_rental(vehicle, days, distance, fuel, season, rules):
    classes = {
        "Car": CarRental,
        "Bike": BikeRental,
        "Truck": TruckRental,
        "Electric": ElectricVehicleRental
    }
    if vehicle not in classes:
        raise InvalidRentalInputError("Vehicle must be Car, Bike, Truck, or Electric")
    return classes[vehicle](days, distance, fuel, season, rules)

def main():
    try:
        rules = load_pricing_rules()
        customer = input("Customer name: ").strip()
        if not customer:
```

```

        raise InvalidRentalInputError("Customer name cannot be empty")

    vehicle = input("Vehicle (Car/Bike/Truck/Electric): ").strip().title()
    days = int(input("Number of days: "))
    distance = float(input("Distance in km: "))
    fuel = input("Fuel (Petrol/Diesel/Electric): ").strip().title()
    season = input("Season (Normal/Peak): ").strip().title()

    rental = create_rental(vehicle, days, distance, fuel, season, rules)
    cost = rental.calculate_cost()

    data = {
        "time": datetime.now().isoformat(timespec="seconds"),
        "customer": customer,
        "vehicle": vehicle,
        "days": days,
        "distance": distance,
        "fuel": fuel,
        "season": season,
        "cost": cost
    }

    save_transaction(data)
    write_receipt(data)
    write_report(data)
    write_log(f"SUCCESS - {customer} rented {vehicle}; cost={cost}")

    print(f"Total rental cost: Rs. {cost:.2f}")
    print("Generated: rental_transactions.csv, receipt.txt, summary_report.json, system_log.txt")

except (ValueError, InvalidRentalInputError, ConfigurationError) as e:
    write_log(f"ERROR - {e}")
    print("Error:", e)
except OSError as e:
    print("File error:", e)

if __name__ == "__main__":
    main()

```

5. Test Case and Generated Outputs

Test Input: Customer = Abhilash, Vehicle = Car, Days = 3, Distance = 150 km, Fuel = Petrol, Season = Peak.

Calculation: $(3 \times 1200 + 150 \times 12) \times 1.10 = (3600 + 1800) \times 1.10 = \text{Rs. } 5940.00$.

Console Output

Total rental cost: Rs. 5940.00
Generated: rental_transactions.csv, receipt.txt, summary_report.json, system_log.txt

receipt.txt

```
VEHICLE RENTAL RECEIPT
-----
Customer : Abhilash
Vehicle   : Car
Days      : 3
Distance  : 150.0 km
Fuel      : Petrol
Season    : Peak
Total     : Rs. 5940.00
```

rental_transactions.csv

```
time,customer,vehicle,days,distance,fuel,season,cost
2026-08-24T03:24:17,Abhilash,Car,3,150.0,Petrol,Peak,5940.0
```

summary_report.json

```
{
  "title": "Vehicle Rental Pricing Engine - Transaction Summary",
  "generated_at": "2026-08-24T03:24:17",
  "transaction": {
    "time": "2026-08-24T03:24:17",
    "customer": "Abhilash",
    "vehicle": "Car",
    "days": 3,
    "distance": 150.0,
    "fuel": "Petrol",
    "season": "Peak",
    "cost": 5940.0
  },
  "status": "success"
}
```

Invalid Input Tests

Input	Expected Result
Days = 0	InvalidRentalInputError: Duration must be greater than 0 days
Distance = -20	InvalidRentalInputError: Distance cannot be negative
Vehicle = Bus	InvalidRentalInputError: Vehicle must be Car, Bike, Truck, or Electric
Missing pricing_rules.csv	ConfigurationError: Pricing file not found
Days = abc	ValueError handled and logged

6. Mark Allocation Mapping and Run Instructions

Requirement	Implementation	Marks
Base class + category subclasses	VehicleRental + CarRental/BikeRental/TruckRental with correct override	10
Configuration / rate loader	pricing_rules.csv + config_loader.py	10
Core processing / read-write / persistence	calculate_cost(), transaction CSV, JSON report	12
Exception handling / validation	InvalidRentalInputError + ConfigurationError + ValueError/OSError handling	8
Output generation	Receipt TXT, system log TXT, transaction CSV, summary JSON	10
Extensibility / evaluation	ElectricVehicleRental + test results	10

How to Run

1. Put main.py, vehicle_rental.py, config_loader.py and pricing_rules.csv in the same folder.
2. Open Command Prompt or Terminal in that folder.
3. Run **python main.py**.
4. Enter the requested values.
5. Check the generated CSV, JSON and TXT output files in the same folder.

Files to Submit

Python source: main.py, vehicle_rental.py, config_loader.py. Configuration: pricing_rules.csv. Documentation: README.txt. Generated outputs: rental_transactions.csv, receipt.txt, summary_report.json, system_log.txt. This PDF can be used as the report/evidence file for the portal upload.

Conclusion: The solution meets all six required sub-tasks, uses only the Python standard library, keeps pricing externally configurable, provides persistent output files, handles errors safely, and can be extended with additional rental categories.